IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Embedding Models and Vector Basics

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How language understanding evolved from rules and ML to LLMs
- Core principles behind Neural Networks, RNNs, and LSTMs
- The role of embeddings, attention, and transformer architectures
- Key terminology and methods used in LLM training (pre-training, fine-tuning, RLHF)
- Real-world significance, limitations, and future directions of LLMs
- Fundamental ML concepts like parameters, tokens, and scaling laws in AI

Topics Covered

- What are Embeddings?
- Token Vectors vs Sentence Embeddings
- Similarity: Cosine vs. Euclidean Distance
- Finding Nearest Neighbours in Vector Space
- Visualising Embeddings
- Vector Search with Embeddings
- Journaling Activity

What are Embeddings?

A bridge between raw data and machine understanding

Why Embeddings: The Machine Understanding Challenge

Raw data problem

Machines cannot directly understand text, images or audio and require numerical input.

Numerical representation

Meaning must be converted into mathematical form that preserves semantics.

Vector space solution

Embeddings represent objects as points in multi-dimensional space reflecting meaning.

Embedding Definition

An embedding is a numerical vector representing data (text, images or audio) where similar items are close and dissimilar ones distant in vector space.

Similar items cluster together
Related concepts naturally group in the same region of the vector space

Distance reflects meaning
Mathematical distance corresponds to semantic similarity or difference

Text Embedding Example

Sentence 1

"I love machine learning"

→ [0.2, 0.8, -0.1, 0.5, ...]

Sentence 2

"Deep learning is amazing"

→ [0.3, 0.7, -0.2, 0.6, ...]

Key Properties of Embeddings

Semantic similarity
Captures meaning and relationships between concepts, beyond surface features like exact word matches.

Fixed length vector
Produces a consistent output size regardless of input length, whether a tweet or a novel.

Learned from data
Discovers optimal representations by training neural networks on large datasets with specific goals

Enable computation
Allow machines to perform mathematical operations like similarity comparison, clustering and classification on complex data.

Types of Embeddings

Word embeddings

- **Examples:** Word2Vec, GloVe, FastText
- Map individual words to dense vectors, capturing semantic relationships like king − man + woman ≈ queen

Sentence embeddings

- **Examples:** Universal Sentence Encoder, SBERT
- Encode entire sentences or paragraphs into fixed-size vectors that preserve contextual meaning.

Types of Embeddings

Image embeddings

- **Examples:** CNN-based feature vectors, ResNet
- Extract visual features from images to enable similarity search and classification.

Multimodal embeddings

- **Examples:** CLIP, DALL-E
- Combine text and image data into the same vector space for cross-modal understanding.

Real-World Applications

Search and Retrieval

- Semantic search finds relevant results without exact keyword matches.
- RAG systems use embeddings to retrieve contextual information.

Real-World Applications

Recommendation Systems

Netflix, Spotify and Amazon use embeddings to suggest content based on user preferences and item similarities.

Real-World Applications

Clustering & Classification

Group similar documents, organise customer segments or automatically categorise content at scale.

Real-World Applications

Similarity detection

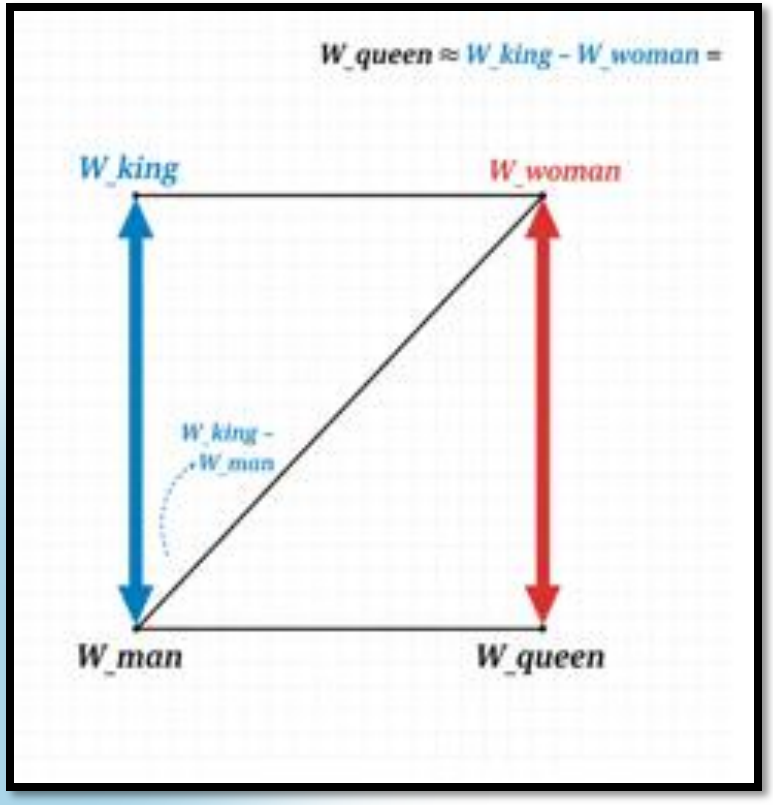
Detect plagiarism, spot fraud and find duplicate images across databases.

Language tasks

Enable machine translation, question answering and conversational AI.

Visualising Embeddings

Word relationships appear when high-dimensional embeddings are projected into 2D space.



- **Semantic relationships** Words like **king** and **queen** cluster together, reflecting consistent links with **man** and woman.

- **Mathematical operations**
  Vector arithmetic reveals analogies:
  king - man + woman ≈ queen

Token Vectors v. Sentence Embeddings

Understanding different levels of text representation

Importance of Text Representation

Machine challenge

Computers process numbers, not words; text must be numerically represented.

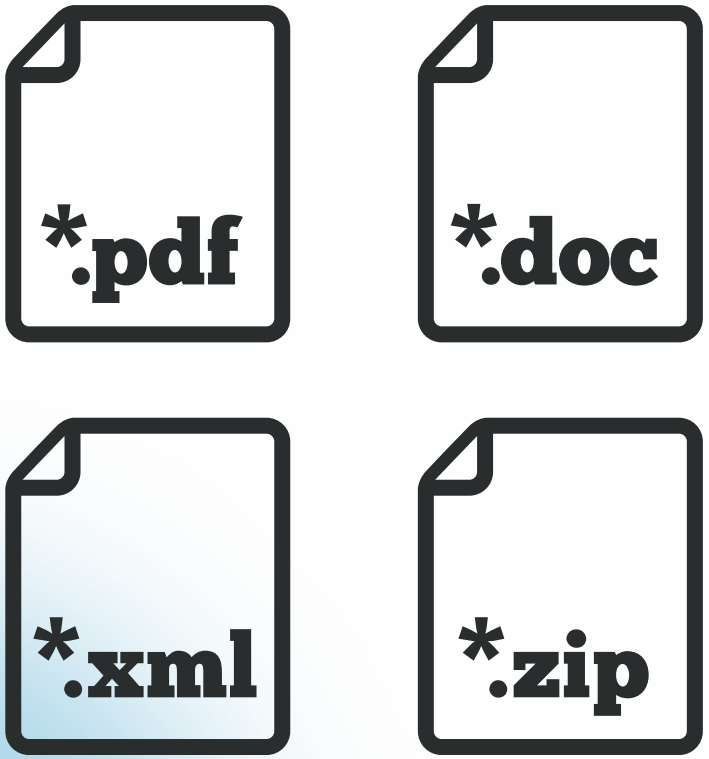
Granularity levels

Text is represented at scales from tokens to full documents.

Task-driven choice

Representation depends on needing word-level detail or document-level meaning.

What are Token Vectors?



- **Definition**:
  - A token is the smallest meaningful unit such as word, subword or character.
  - Each token maps to a high-dimensional vector (embedding).
- **Example breakdown**:
  - “Cats are cute” becomes three vectors: [v1, v2, v3].
  - Essential for LLMs and NLP tasks where sequence and word order matter.

Token vectors preserve word meaning and positional relationships.

Properties of Token Vectors

Each vector captures the semantic meaning of its word or subword.

Token vectors adapt to surrounding words in transformer models.

Preserve word order and relationships for positional awareness.

Individual word meaning

Context-sensitive

Sequential processing

Primary Use Cases of Token Vectors

- Language modelling and text generation
- Machine translation with word-level alignment
- Named Entity Recognition (NER) and part-of-speech tagging

What are Sentence Embeddings?

Unified representation
Compress sentences or documents into fixed-length vectors capturing overall meaning.

Consistent output
Produce same-sized vectors regardless of sentence length,

e.g. “Cats are cute” → [0.23, -0.11, 0.87, ...].

This approach enables direct comparison between texts of different lengths and facilitates efficient similarity calculations.

Properties of Sentence Embeddings

Global context
Captures holistic meaning and semantic intent of entire text

Direct comparison
Enables straightforward similarity calculations using vector distances

Efficiency
Provides compact representation for efficient storage and processing

Key Applications of Sentence Embeddings

- Semantic search and information retrieval
- Document clustering and categorisation
- Text similarity and recommendation systems

Comparison

Token Vectors v. Sentence Embeddings

Token Vectors	Sentence Embeddings
Represent multiple points—one per word, forming a sequence cloud	Represent a single point summarising the entire sentence's meaning

Think of token vectors as individual ingredients and sentence embeddings as the dish's overall flavour.

Concrete Example

Input sentence: "The cat is sleeping on the sofa."

Token Vectors (Individual Words)

```
the      → [0.12, -0.05, 0.78]
cat      → [0.21, 0.34, -0.15]
is       → [-0.33, 0.67, 0.22]
sleeping → [0.45, -0.12, 0.89]
on       → [0.17, 0.23, -0.44]
the      → [0.12, -0.05, 0.78]
sofa     → [0.45, -0.22, 0.67]
```

Sentence Embedding (Whole Meaning)

```
entire sentence →[0.12,
0.55, -0.09, 0.78,  -
0.23, 0.44, 0.67, ...]
```

Single vector captures the scene of a cat resting on furniture.

When to Use What?

Use token vectors when

Word-level precision is critical for your task

- Named entity recognition
- Part-of-speech tagging
- Machine translation
- Text generation

Use sentence embeddings when

Overall semantic meaning is what matters most

- Semantic search systems
- Document clustering
- Question-answering retrieval
- Content recommendation

Pro tip: Modern systems combine token vectors for detailed analysis with sentence embeddings for efficient retrieval and comparison.

Similarity: Cosine vs. Euclidean Distance

How machines measure closeness in vector space

What is Similarity?

A way to quantify
closeness between vectors

High similarity → vectors represent
semantically similar items

Different measures → different
notions of “closeness”

Why Similarity Matters

Vector space

Embeddings transform data into high-dimensional vector representations, creating a mathematical space where relationships can be measured.

Similarity measures

To compare embeddings effectively, we need robust methods to quantify how close or similar vectors are to each other.

Two champions

Cosine Similarity and **Euclidean Distance** are the most widely used methods in machine learning applications.

Understanding Similarity

What is Similarity?

Measures similarity or difference between vectors in multidimensional space

High similarity scores reflect semantically related items or concepts

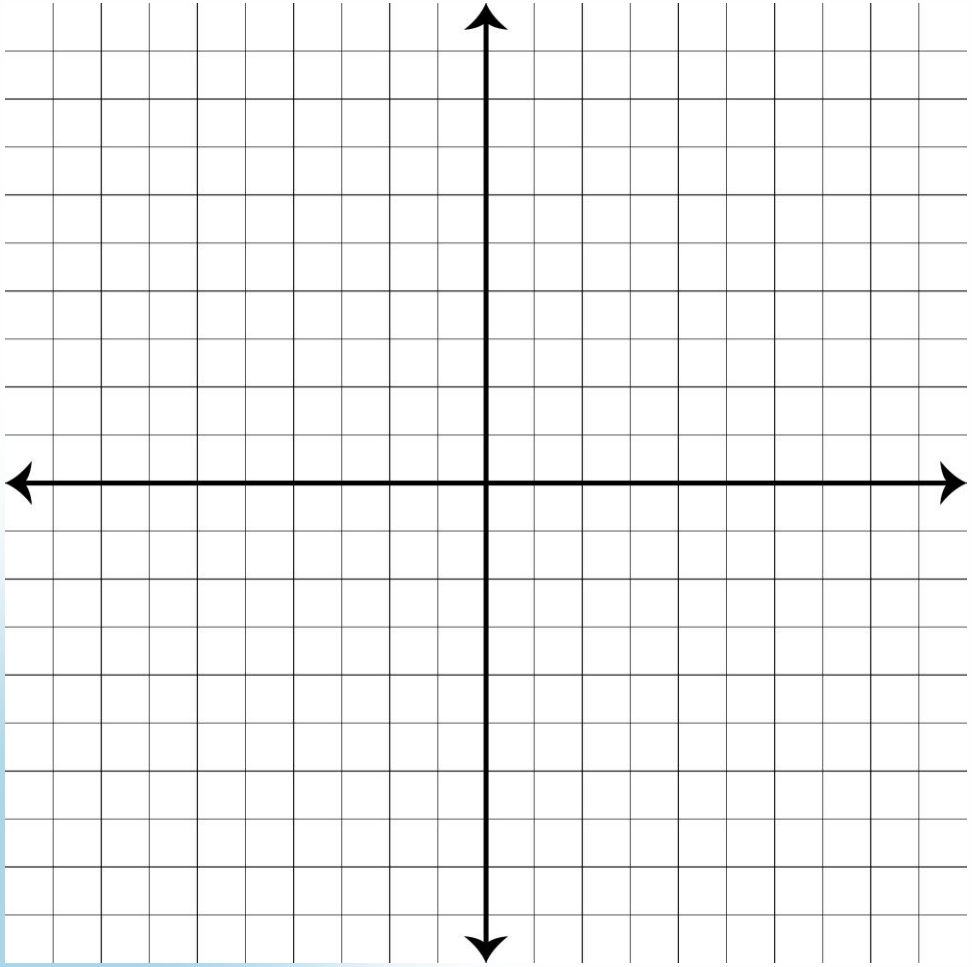
Different similarity measures capture different aspects of "closeness" based on your specific use case

Quantifying closeness

Semantic relationships

Multiple perspectives

Euclidean Distance



- **Definition**
  The straight-line distance between two points in space - exactly like measuring with a ruler in physical space

- **Mathematical formula**

$$d(A,B)=\sqrt{\sum_i (A_i - B_i)^2}$$

- **Core intuition**
  "How far apart are the vectors?" - Sensitive to both direction and magnitude of vectors

Cosine Similarity

- **Definition**
  Measures the angle between vectors, completely ignoring their magnitude or length in the vector space

- **Mathematical formula**

$$cos(\theta) = \frac{A\ B}{\|A\|}$$

- **Core intuition**
  "How aligned are the vectors?" - Values range from -1 to 1, where 1 means identical direction

Word Embedding Example

"King" = [0.8, 0.5]
"Queen" = [0.82, 0.52]

Small valuePoints are close in space

Near 1.0Same direction = similar meaning

Vector representations

Euclidean distance

Cosine similarity

Both metrics detect semantic similarity, each through a different mathematical approach.

When to Use Euclidean Distance

Image processing
Compares pixels where absolute colour and intensity differences matter.

Clustering analysis
Groups numerical features considering both scale and direction

Geometric Problems
Analyses spatial data where physical distance measurements are key.

Best use: Ideal when absolute magnitude and scale differences matter to the task

When to Use Cosine Similarity

Focuses on meaning and context over embedding magnitude in NLP applications

Compares document orientation to indicate topical similarity, regardless of length

Matches user preferences by aligning vector directions for similar tastes

Semantic search

Document similarity

Recommender systems

Best use: Ideal when semantic meaning and direction matter more than magnitude.

Feature Comparison

Aspect	Euclidean distance	Cosine similarity
Primary focus	Magnitude + Direction	Direction only
Scale sensitivity	Yes	No (Normalised)
Value range	$0 \rightarrow \infty$	$-1 \rightarrow 1$
Best use case	Raw numeric comparison	Semantic similarity (NLP)
Computational cost	Lower	Higher (Normalisation)

Demo: Hands-On Gensim Word to Vector

Similarity Search: Finding Nearest Neighbours in Vector Space

Core Concept: Vector Space Navigation

Embeddings	Similarity measures	Nearest neighbours
Converts data into high-dimensional vectors capturing meaning and relationship	Quantifies closeness using functions like cosine similarity and Euclidean distance	Identifies items closest in distance or highest in similarity to the query vector

Search Methods: From Simple to Scalable

Brute Force Search

- Compare query against every item in database
- Guaranteed exact results but computationally expensive
- O(n) complexity – does not scale well

Approximate Nearest Neighbour (ANN)

- Trade-off accuracy for speed using indexing structures
- Methods like HNSW, LSH and product quantisation
- Essential for large-scale production systems

Real-World Applications of Similarity Search

Information retrieval

Performs semantic search that understands meaning beyond keywords, powering search engines.

Recommender systems

Finds similar users or items to personalise recommendations (“people who liked this also liked…”).

Real-World Applications of Similarity Search

Fraud & anomaly detection

Identifies unusual patterns by spotting data points without close neighbours.

Image & video search

Uses visual similarity for reverse image search and content-based retrieval

RAG systems

Supports LLMs by retrieving relevant documents or passages for context

Visualising Embeddings

Using PCA and t-SNE to understand vector spaces

The Visualisation Challenge

High-Dimensional reality
Embeddings exist in hundreds to thousands of dimensions, beyond human perception.

Human limitation
Humans interpret data effectively only in 2D or 3D space.

Dimension reduction
Mathematical techniques project high-dimensional data into lower, interpretable dimensions.

PCA: Principal Component Analysis

Linear Technique
Projects data onto principal components—directions of maximum variance

Key strengths
Fast computation, interpretable, preserves global structure

Best use cases
Quick visualisation, trend detection, understanding data distribution.

PCA in Action: Word Relationships

Projects word vectors into 2D, clustering semantically related words meaningfully.

Words like king, queen, man, woman reveal clear linear patterns in embeddings.

Key Insight

PCA excels at highlighting linear relationships and broad embedding patterns.

t-SNE: Non-Linear Clustering

t-distributed Stochastic Neighbour Embedding

- Excels at revealing hidden clusters
- Preserves local relationships
- More computationally intensive
- Better for fine-grained analysis

A sophisticated non-linear method that preserves local neighbourhood structure during dimensionality reduction.

Technique Comparison

Aspect	PCA	t-SNE
Type	Linear reduction	Non-linear reduction
Preserves	Global structure	Local clusters
Speed	Fast	Slower
Interpretability	High	Harder to interpret
Best for	Broad trends	Fine-grained clusters

Choose your technique based on whether you need to understand global patterns (PCA) or discover local clustering structures (t-SNE).

Visualisation Workflow

Generate embeddings
Create word or sentence embeddings using models like Word2Vec, BERT or Sentence-BERT.

Choose reduction method
Select PCA for global structure or t-SNE for cluster discovery based on goals.

Project to lower dimensions
Reduce embeddings to 2D or 3D using the chosen technique.

Create scatter plots
Visualise the projected embeddings using interactive, colour-coded scatter plots

Interpret results
Analyse clusters, relationships and patterns to extract insights.

Practical Applications

Semantic relationships
Understand how words and concepts connect within your embedding model

Embedding quality
Assess whether embeddings capture meaningful semantic links

Document analysis
Explores clusters and uncover hidden topics in document collections

Education & Demo
Make abstract embedding concepts tangible for teaching and presentations

Vector Search with Embeddings

From cosine similarity to practical applications

Why Traditional Search Falls Short

Traditional search

Relies on exact keyword matching, literal text comparison and misses context and meaning.

The problem

Loses semantic similarity; e.g., "doctor" ≠ "physician," "car" ≠ "automobile."

Vector search solution

Uses semantic search with embeddings to understand meaning and find related concepts.

Real-World Example: Query vs Documents

Query:

"healthy recipes with tofu"

Document 1

"**Best ways to cook tofu"**
High similarity - Same topic, different words

Document 2

"**Fast food near me"**
Low similarity - Completely different intent

Four Steps to Vector Search

Convert text to vectors using OpenAI Embeddings API

Calculate cosine similarity between query and documents

Generate embeddings → **Store vectors** → **Compute similarity** → **Return results**

Index high-dimensional vectors in FAISS database

Retrieve k-nearest neighbours as ranked search results

OpenAI Embeddings: Text to Vectors

What it does

API converts text into high-dimensional vectors capturing semantic meaning.

Example Embeddings:

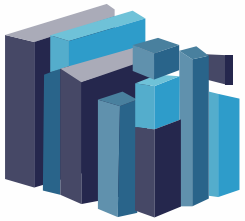
"cat" → [0.12, -0.34, 0.89, ...]"dog" → [0.13, -0.36, 0.88, ...]
Similar animals produce similar vector values.

Why it works

Related concepts map to nearby points in vector space.

FAISS: Scaling Vector Search

Inpuge input



Flagiure extraocting

Facebook AI Similarity Search

Feature Extraction

Search quries

Efficient Library

Facebook's open-source solution for similarity search at massive scale

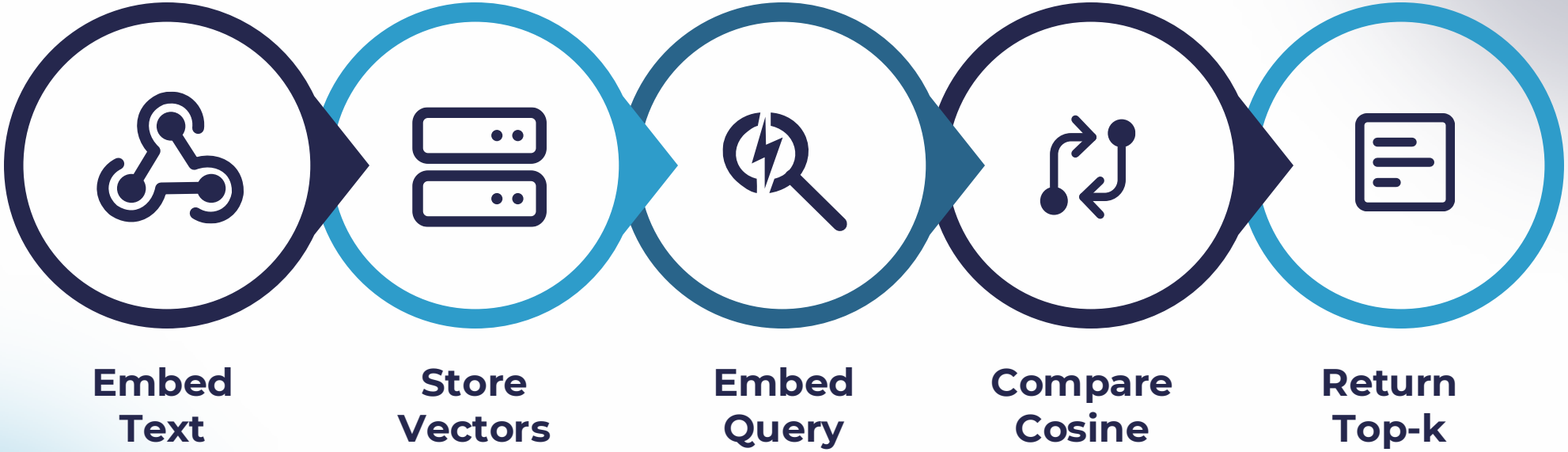
Billion-scale support

Handles billions of vectors with optimised indexing and search algorithms

Nearest neighbour search

Finds most similar vectors quickly using approximate algorithms

Complete Vector Search Workflow



This workflow powers semantic search across millions of documents in real-time.

Search in Action

Sample Query:

"AI tools for productivity"

Query processing

Text converted to embedding vector

Vector comparison

FAISS finds 3 most similar document vectors

Relevant results

Returns docs about AI productivity applications, automation tools and workflow optimisation

Journaling Activity

Reflection Time

> **"How can vector search improve workflows in your domain?"**

Real-World Applications

Healthcare

Retrieve patient notes with similar symptoms or treatments, even if described differently

Education

Find learning resources by concept, not just keywords

Business Intelligence

Discover relevant past project documents using semantic similarity.

Q & A

Thank you